Digital Signal Processing – Assignment 2
FIR Filters

William Chambers (GUID: 2906463C)
Saad Aziz (GUID: 2917444A)

University of Glasgow, School of Engineering

November 8, 2026

1 Recordings

[State who recorded the ECGs in the lab, the electrode placement/lead setup, the sampling rate f_s (calculated from column 0 timestamps) and corresponding Nyquist frequency $f_s/2$, which column of the TSV contains the ECG channel (column 8, 0-indexed), and why each partner's recording was selected (e.g. primary recording for Tasks 1–4 vs. partner's recording for the Task 5 matched-filter template).]

2 Task 1: Time and Frequency Analysis

[Briefly state how the time axis and single-sided linear frequency spectrum ($X = \frac{1}{N}\,\mathrm{FFT}(x)$ up to $f_s/2$) were computed from the raw recording without using prohibited functions like `fftfreq`. Emphasise that the frequency domain is plotted on a **linear axis** as required by the brief.]

[Insert vector Figure (PDF/EPS/SVG): Raw ECG in the time domain showing (1) the full recording to reveal overall baseline drift/offset, and (2) a zoomed-in trace of a single heartbeat annotated (e.g. via Inkscape) with the P, Q, R, S, and T waves and proper axis labels/units.]

Figure 1: [Caption specifying the recording, full time-domain trace, and zoomed single heartbeat with labelled PQRST waves.]

[Insert vector Figure (PDF/EPS/SVG): Raw ECG in the frequency domain on **linear axes** (both full magnitude view and zoomed-in magnitude view if needed to resolve smaller peaks), annotated (e.g. via Inkscape) to point out DC/baseline wander, heart-rate fundamental and harmonics, 50 Hz mains interference, and higher-frequency noise floor.]

Figure 2: [Caption describing the annotated linear-axis frequency spectrum of the raw ECG.]

ECG signal and harmonics. [Measure the beat-to-beat (R–R) period from the time-domain plot, calculate the corresponding fundamental heart rate frequency $f_0 = 1/T_{\mathrm{RR}}$, locate the fundamental peak and its integer harmonics ($2f_0, 3f_0, \ldots$) in the zoomed frequency spectrum, and identify the frequency band where the diagnostic PQRST energy resides.]

Noise and interferences. [Identify and explain every non-ECG component in both domains: (1) DC offset and low-frequency baseline wander (respiration/movement drift in time domain, large spike near 0 Hz in spectrum), (2) 50 Hz mains power-line hum (and state whether any

100 Hz harmonic is present below the 125 Hz Nyquist limit), and (3) broadband muscle (EMG) / electronic noise floor and where it overlaps or sits above the ECG spectrum.]

Frequencies to remove. [Explicitly list the exact frequency ranges chosen for removal (in Hz) and justify **why** each cut-off was chosen using your harmonic analysis: why the low cut-off removes DC and baseline wander without attenuating the fundamental heart rate f_0, and why the upper bandstop/lowpass cut-off removes 50 Hz mains hum (and high-frequency noise) without rounding or distorting the sharp QRS complex.]

3 Task 2: FIR Filter Coefficient Design

Number of coefficients. [Give the exact mathematical formula and reasoning for how the number of taps M is automatically calculated inside your general-purpose design function from f_s and the array of frequency ranges to remove (e.g. finding the narrowest stopband width $\Delta f_{\min}$, setting the required frequency resolution / bin width δf, computing $M = f_s/\delta f$, and enforcing even parity if required for symmetric half-swapping). Discuss the engineering trade-off between frequency resolution (sharper cut-offs with larger M) versus filter delay ($M/2$ samples) and realtime computational cost.]

IFFT coefficient calculation. [Explain step-by-step how the general-purpose function computes the FIR coefficients: (1) constructing the ideal frequency response array $H[k]$ of length M with passbands set to 1 and target stopband ranges set to 0, (2) maintaining conjugate/Hermitian symmetry across positive and negative frequencies and explicitly handling edge cases such as DC (0 Hz, bin 0) and Nyquist ($f_s/2$), (3) applying the IFFT and taking the real part to obtain the impulse response $h[n]$, (4) shifting/swapping the two halves of $h[n]$ so the impulse response is causal and centred at $M/2$ rather than split across the ends, and (5) multiplying by a window function (e.g. Hamming) to suppress Gibbs phenomenon ringing caused by truncating the impulse response.]

[Insert vector Figure (PDF/EPS/SVG): Plot of the designed FIR filter showing (1) the final shifted and windowed impulse response / tap coefficients $h[n]$ against tap index n, and (2) the resulting frequency response magnitude $|H(f)|$ verifying the stopbands at DC/baseline wander and 50 Hz.]

Figure 3: [Caption stating the calculated number of taps M, the removed frequency ranges, the window used, and what the time/frequency plots show.]

4 Task 3: Realtime FIR Filtering

Implementation. [Explain the architecture of `class FIRfilter` with constructor `__init__(self, _coefficients)` and method `dofilter(self, v)`. Emphasise that `dofilter` accepts a single scalar input sample $x[n]$ and returns a single scalar output sample $y[n] = \sum_{m=0}^{M-1} h[m]\, x[n-m]$ to simulate sample-by-sample realtime processing. Explicitly explain how the delay line is implemented efficiently (e.g. using a circular ring buffer and a moving index pointer/offset rather than shifting all M array elements at every time step, which the brief warns loses marks).]

Filtered ECG results. [Show the result of feeding the raw ECG sample-by-sample through `dofilter`. Explain the initial transient / buffer-fill period (M samples) and the linear-phase group delay ($M/2$ samples). Verify and explicitly comment on: (1) complete removal of DC offset and baseline wander in the full time trace, (2) removal of the 50 Hz hum in the time and

frequency domains, and (3) visual confirmation on a zoomed single heartbeat that the P, Q, R, S, and T waves are intact, sharp, and undistorted.]

[Insert vector Figure (PDF/EPS/SVG): Task 3 realtime FIR filtered ECG showing (1) the full filtered time-domain trace (flat baseline around 0 V), (2) a zoomed-in single heartbeat annotated with P, Q, R, S, T to prove the morphology is intact and undistorted, and (3) the linear frequency spectrum of the filtered ECG confirming the stopbands.]

Figure 4: [Caption describing the sample-by-sample FIR filtered ECG in the time domain (full and zoomed single beat with PQRST) and frequency domain.]

5 Task 4: Adaptive LMS Filter

Implementation. [Explain how `doFilterAdaptive(self, signal, noise, learningRate)` is added to `FIRfilter` and operates sample-by-sample on scalars: (1) what the primary input `signal` $d[n]$ is (and if DC/baseline wander is removed beforehand), (2) how the scalar 50 Hz reference `noise` $x[n] = \sin(2\pi \cdot 50\, n/f_s)$ is generated and filtered by the adaptive weights to estimate the interference $y[n]$, (3) how the cleaned ECG error output $e[n] = d[n] - y[n]$ is computed, and (4) how each filter weight w_k is updated at every step ($w_k \leftarrow w_k + \mu\, e[n]\, x[n-k]$). Justify your choice of filter length M_{LMS} and learning rate μ (trade-off between adaptation speed and stability/excess error).]

Comparison with FIR filter design. [Compare the adaptive LMS filter output directly against the Task 3 fixed FIR bandstop filter: compare (1) 50 Hz noise suppression quality and PQRST waveform preservation on a zoomed heartbeat, (2) startup behaviour (LMS weight convergence time vs. fixed FIR buffer fill and group delay), (3) computational efficiency (number of taps required for LMS vs. the large M needed for the fixed FIR filter), and (4) frequency selectivity (narrow notch tracking the reference vs. wider fixed bandstop).]

[Insert vector Figure (PDF/EPS/SVG): Direct visual comparison of the Task 4 adaptive LMS output and the Task 3 fixed FIR output, including (1) full or startup time trace showing LMS convergence, (2) zoomed single heartbeat overlay showing PQRST fidelity of both methods, and (3) frequency spectrum comparison if applicable.]

Figure 5: [Caption comparing the adaptive LMS filter result (μ, M_{LMS}) with the Task 3 fixed FIR filter design.]

6 Task 5: ECG Heartbeat Detection

Matched filter template. [Explain how a single clean PQRST heartbeat was isolated from the **other lab partner's** filtered ECG recording to act as a real ECG template. Explain the theory and implementation of the matched filter: why the template array is time-reversed ($h[n] = \text{template}[N-1-n]$) and passed into a `FIRfilter` instance so that sample-by-sample convolution via `dofilter` performs cross-correlation in realtime without using prohibited functions like `correl` or `conv`. Mention any post-processing applied to the matched filter output (e.g. squaring) to enhance the R-peak SNR.]

[Insert vector Figure (PDF/EPS/SVG): The single heartbeat template extracted from the other lab partner's ECG (and/or its time-reversed matched filter impulse response) with proper time and amplitude labels.]

Figure 6: [Caption identifying which partner's recording the template came from, the time window extracted, and the resulting matched filter coefficients.]

R-peak detection and momentary heart rate. [Explain how R-peaks are detected from the matched filter output (e.g. threshold crossing / local peak detection) and how the momentary heart rate is calculated from the inverse intervals between consecutive R-peaks ($\mathrm{HR} = 60/\Delta t_{\mathrm{RR}}$ in bpm). Explicitly detail the heuristic(s) implemented to reject false detections and artefacts (e.g. minimum refractory period corresponding to a maximum physiological heart rate of $\leq 200\,\mathrm{bpm}$, minimum heart-rate bound, and/or ignoring the initial filter startup transient) and discuss the resulting heart rate trace over time.]

[Insert vector Figure (PDF/EPS/SVG): (1) Matched filter output (and/or ECG trace) showing the detected R-peaks, and (2) the momentary heart rate (in bpm or bps/Hz, clearly labelled) plotted against time after heuristic filtering.]

Figure 7: [Caption describing the R-peak detections and the momentary heart rate vs. time plot with heuristic outlier removal.]

A Acknowledgement of GenAI Use

Tools: [List every GenAI tool used with its exact name, version, and publisher (e.g. Claude Opus 5.5 (Anthropic); Gemini (Google); GitHub Copilot (Microsoft)) as required by the brief and UoG guidance.]
Use: [State clearly (1) how each GenAI tool was used (e.g. LaTeX report framework, debugging, syntax lookups, code optimisation), and (2) explicitly what work you contributed yourself (e.g. lab ECG recordings, spectral analysis, filter design logic, ring-buffer and LMS implementations, matched filter and heuristic design, and writing the report).]

B Usage instructions

[Provide exact platform-independent command-line instructions to run the code from a terminal (not Spyder) after unzipping the submission, list required Python packages (`numpy`, `scipy`, `matplotlib`), and confirm that running the script(s) displays all plots without crashing and uses only relative paths.]

C Python Code

[Before submitting, verify that: (1) every Python file in the submission is included below, (2) no prohibited high-level functions (`fftfreq`, `lfilter`, `firwin`, `conv`, `correl`, FFT/IIR filtering, or array-in/array-out a-causal filtering) appear anywhere, (3) no absolute file paths exist, and (4) any LLM-generated test/debug bloat is removed or optimised so every line is concise, human-readable, and explainable in an interview.]

C.1 support_funcs.py

```python
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt

BASE_DIR = Path(__file__).resolve().parent


def import_ecg_data(filename):
    """
    Parameters:
    filename: str or Path
        The name of the file containing the ECG data.

    Returns:
    tuple[np.ndarray, np.ndarray, float]
        (t, ecg, sample_rate) - time vector (s), ECG signal (V), and sampling rate (Hz).
    """
    ECG_COLUMN = 8  # ADC channel 2
    data = np.loadtxt(BASE_DIR / filename)
    t = data[:, 0]
    ecg = data[:, ECG_COLUMN]
    sample_rate = 1 / (t[1] - t[0])  # fs = 250 Hz

    return t, ecg, sample_rate


def vector_plot_export(savename):
    """
    Exports the current matplotlib figure as vector graphics (.svg and .pdf) to Report/figs/.

```

```
    Parameters:
    savename: str or Path
        Output file name (without extension).
    """
    figs_dir = BASE_DIR / "Report" / "figs"
    figs_dir.mkdir(parents=True, exist_ok=True)
    stem = Path(savename).stem
    plt.savefig(figs_dir / f"{stem}.svg", format="svg", bbox_inches="tight")
    plt.savefig(figs_dir / f"{stem}.pdf", format="pdf", bbox_inches="tight")
```

C.2 ecg_plot.py

```
# Task 1

import numpy as np
from scipy.fft import fft
import matplotlib.pyplot as plt

from support_funcs import import_ecg_data

# Task 1 Plots
# Function to plot ECG signal in the time and frequency domain
def plot_ecg(name, sample_rate, ecg, beat_start, beat_end):
    n = len(ecg)
    time = np.arange(n) / sample_rate # array of times

    # Frequency domain (linear)
    ecg_fft = (1 / n) * fft(ecg)
    mag_fft = np.abs(ecg_fft)
    freqs = np.arange(n // 2) * sample_rate / n # frequency bins up to sample_rate/2

    plt.figure(figsize=(12, 8))
    plt.suptitle(f"ECG Analysis: {name}")

    # fig 1: time domain
    plt.subplot(2, 1, 1)
    plt.title("Time domain")
    plt.plot(time, ecg)
    plt.xlim(0, n / sample_rate)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")

    # zoomed in
    plt.subplot(2, 1, 2)
    plt.title("Zoomed: Two Peaks")
    plt.plot(time, ecg)
    plt.xlim(beat_start, beat_end)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")

    plt.tight_layout() # stop axes overlapping

    # fig 2: frequency domain
    plt.figure(figsize=(12, 8))
    plt.suptitle(f"ECG spectrum: {name}")

    plt.subplot(2, 1, 1)
    plt.title("Full spectrum")
    plt.plot(freqs[0:], mag_fft[0:n // 2])
    plt.xlim(0, sample_rate / 2)
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (V)")

```

```
    # zoomed in
    plt.subplot(2, 1, 2)
    plt.title("Zoomed-in Spectrum")
    plt.plot(freqs[0:], mag_fft[0:n // 2])
    plt.xlim(0, sample_rate / 2)
    plt.ylim(0, 3e-5) # reveals the small peaks
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (V)")

    plt.tight_layout() # stop axes overlapping

# import ECG Data & plot
t, ecg, sample_rate = import_ecg_data("ecg1.tsv")
plot_ecg("ECG 1", sample_rate, ecg, 5.5, 6.7) # zoom window containing two beats
plt.show()
```

C.3 fir_design.py

```
#Task 2
import numpy as np
from scipy.fft import ifft

#This function determines the FIR coefficients for freq. ranges specified in the function arguments (2D array: [low, high]) and a given sample rate.
def fir_design(sample_rate, ranges):

    ranges = np.array(ranges)
    min_width = np.min(ranges[:, 1] - ranges[:, 0])

    bin_size = min_width / 2
    M = int(sample_rate / bin_size) #half the narrowest stopband, so every stopband covers at least 2 bins

    M += M%2 # ensure M is even

    #Using folded axis to create a symmetric frequency response
    f = np.arange(M) * sample_rate / M #freq. of each bin
    f = np.minimum(f, sample_rate - f) #fold: bin M-k gets the same freq. as bin k
    H = np.ones(M) #initialise frequencies to 1 (pass band)

    #define low and high, setting the bins within this band to 0.
    for low, high in ranges:
        H[(f >= low) & (f <= high)] = 0 #remove bins in range (set to 0)

    # IFFT - produces the impulse response, keeping only the real part (H is mirrored, so complex cancels)
    h = np.real(ifft(H))

    # Swap the two halves: centres the impulse response at M/2
    h_shift = np.zeros(M)
    h_shift[0:M//2] = h[M//2:M]
    h_shift[M//2:M] = h[0:M//2]

    h_windowed = h_shift * np.hamming(M) #apply hamming window to reduce Gibbs ripple from truncation

    #Edge cases (DC and Nyquist) are covered by the folded axis (bin 0 and bin M/2 [fs/2])

    return h_windowed
```

C.4 fir_filter.py

```python
# task 3a + 4a - FIRfilter class functions
import numpy as np


class FIRfilter:
    def __init__(self, _coefficients):
        """
        Parameters:
        _coefficients: array
            The M filter coefficients calculated in Task 2.
        """
        self.coefficients = np.array(_coefficients, dtype=float) # load coefficients into object
        self.ntaps = len(self.coefficients) # get buffer array length

        self.buffer = np.zeros(self.ntaps, dtype=float) # generate buffer array

        self.offset = 0 # ring buffer index where the next sample is written

    def dofilter(self, v):
        """
        Parameters:
        v: float
            The incoming input sample x[n].

        Returns:
        float
            The filtered output sample y[n].
        """
        self.buffer[self.offset] = v # load current sample into buffer

        k = self.offset + 1
        # output = sum[0:M-1](h(m) \dot x(n-m))
        output = (
            np.dot(self.coefficients[:k], self.buffer[self.offset::-1])  # fwd sum
            + np.dot(self.coefficients[k:], self.buffer[:self.offset:-1]) # backward sum
        ) # dot product multiply coefficients with samples in object

        # iterate offset position, reset to zero
        if self.offset == self.ntaps - 1:
            self.offset = 0
        else:
            self.offset += 1


        return float(output)

    def doFilterAdaptive(self, signal, noise, learningRate):
        """
        Parameters:
        signal: float
            Current contaminated ECG sample d[n].
        noise: float
            Current reference noise sample x[n] (e.g. sin(2*pi*50/fs * n)).
        learningRate: float
            LMS step size (mu) controlling how fast the weights adapt.

        Returns:
        float
            The cleaned ECG sample e[n] = d[n] - y[n].
        """
        read_idx = self.offset # store current write index before dofilter increments offset

        y = self.dofilter(noise) # filter reference noise x[n] to estimate interference y[n]
```

```
64
65          e = signal - y # subtract estimated noise from signal to get error / clean ECG e[n]
66
67          k = read_idx + 1
68          step = learningRate * e # calculate LMS weight update factor (mu * e[n])
69          # w_m[n+1] = w_m[n] + mu * e[n] * x[n-m]
70          self.coefficients[:k] += step * self.buffer[read_idx::-1] # fwd tap weight update
71          self.coefficients[k:] += step * self.buffer[:read_idx:-1] # backward tap weight update
72
73          return float(e)
```

C.5 fir_realtime_sim.py

```
1  # Task 3b - Simulating realtime FIR filtering importing fir_design and fir_filter.
2
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from scipy.fft import fft
6
7  from fir_design import fir_design
8  from fir_filter import FIRfilter
9  from support_funcs import import_ecg_data
10
11 # FIR Filter params from Task 2 (also used by heartbeat.py)
12 BASELINE_WANDER = [0, 0.5]  # Remove DC and low-frequency baseline drift
13 MAINS_AND_HF_HUM = [45, 125]  # Remove 50 Hz mains hum and high-frequency noise up to Nyquist (
       fs/2)
14
15 def run_fir_filter(ecg, sample_rate):
16     # generate list of tap coeffs
17     h = fir_design(sample_rate, [BASELINE_WANDER, MAINS_AND_HF_HUM])
18
19     # create object and generate output list
20     fir = FIRfilter(h)
21     ecg_filtered = []
22
23     for sample in ecg:
24         ecg_filtered.append(fir.dofilter(sample))
25
26     # convert list into np to plot
27     ecg_filtered = np.array(ecg_filtered)
28     group_delay = (len(h) // 2) / sample_rate
29     print(f"FIR filter: M = {len(h)} taps, group delay = {group_delay:.1f} s")
30
31     return ecg_filtered, group_delay
32
33
34 if __name__ == "__main__": # since its being imported we need to prevent plotting repeats
35     # load ECG Data
36     t, ecg, sample_rate = import_ecg_data("ecg1.tsv")
37
38     ecg_filtered, group_delay = run_fir_filter(ecg, sample_rate)
39     n = len(ecg_filtered)
40
41     # generate plots
42     # Filtered ECG (Time Domain, Zoomed Heartbeat, Frequency Spectrum) ---
43     freqs = np.arange(n // 2) * sample_rate / n
44     raw_fft = (1 / n) * np.abs(fft(ecg))
45     filt_fft = (1 / n) * np.abs(fft(ecg_filtered))
46
47     beat_start, beat_end = 5.5, 6.7
48     in_beat = (t >= beat_start) & (t <= beat_end)
49
```

```
    plt.figure(figsize=(12, 8))
    plt.suptitle("Task 3: Realtime FIR Filtered ECG: ECG 1")

    # Full time domain
    plt.subplot(3, 1, 1)
    plt.title("Time Domain: Raw vs Filtered ECG")
    plt.plot(t, ecg, alpha=0.5, label="Unfiltered ECG")
    plt.plot(t, ecg_filtered, label="Filtered ECG")
    plt.xlim(0, n / sample_rate)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")
    plt.legend(loc="upper right")
    plt.grid()

    # Zoomed single heartbeat (PQRST check)
    plt.subplot(3, 1, 2)
    plt.title("Zoomed Heartbeat (PQRST Complex): Raw vs Filtered ECG")
    plt.plot(t, ecg - np.mean(ecg[in_beat]), alpha=0.5, label="Unfiltered ECG (DC mean
    subtracted)")
    plt.plot(t - group_delay, ecg_filtered, label=f"Filtered ECG (delay-aligned by {group_delay
    :.1f} s)")
    plt.xlim(beat_start, beat_end)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")
    plt.legend(loc="lower right")
    plt.grid()

    # Linear frequency spectrum
    plt.subplot(3, 1, 3)
    plt.title("Frequency Spectrum: Raw vs Filtered ECG")
    plt.plot(freqs, raw_fft[:n // 2], alpha=0.5, label="Unfiltered Spectrum")
    plt.plot(freqs, filt_fft[:n // 2], label="Filtered Spectrum")
    plt.xlim(0, sample_rate / 2)
    plt.ylim(0, 3e-5)
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (V)")
    plt.legend(loc="upper right")
    plt.grid()

    plt.tight_layout()
    plt.show()
```

C.6 lms_filter.py

```
# Task 4b - Simulating realtime adaptive LMS filtering importing fir_filter.

import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

from fir_filter import FIRfilter
from fir_realtime_sim import run_fir_filter
from support_funcs import import_ecg_data

# Load ECG Data
t, ecg, sample_rate = import_ecg_data("ecg1.tsv")

# Run Task 3 FIR filter for comparison
ecg_fir, group_delay = run_fir_filter(ecg, sample_rate)

# Adaptive LMS Filter Setup: two cancellers in series

# DC / baseline wander canceller (constant reference = 1.0)
```

```
M_DC = 1 # number of taps
mu_dc = 0.0125 # learning rate
lms_dc = FIRfilter(np.zeros(M_DC)) # adaptive filter, weights start at 0

# 50 Hz mains canceller (50 Hz sine reference)
M_lms = 10 # number of taps
mu_lms = 0.01 # learning rate
f_noise = 50.0 # ref signal frequency
lms = FIRfilter(np.zeros(M_lms)) # adaptive filter, weights start at 0

# Realtime Adaptive Filtering
ecg_lms = [] # results list

for i, sample in enumerate(ecg):

    # remove DC / baseline wander
    ecg_no_dc = lms_dc.doFilterAdaptive(sample, 1.0, mu_dc)
    # remove 50 Hz powerline interference
    noise_50 = np.sin(2 * np.pi * f_noise * i / sample_rate)
    ecg_lms.append(lms.doFilterAdaptive(ecg_no_dc, noise_50, mu_lms))

ecg_lms = np.array(ecg_lms) # convert to np array to plot
n = len(ecg_lms)

# Plotting (FIR Filtered ECG vs Adaptive LMS Filtered ECG)
freqs = np.arange(n // 2) * sample_rate / n
fir_fft = (1 / n) * np.abs(fft(ecg_fir))
lms_fft = (1 / n) * np.abs(fft(ecg_lms))

beat_start, beat_end = 5.5, 6.7

plt.figure(figsize=(12, 8))
plt.suptitle("Task 4: Realtime Adaptive LMS vs FIR Filtered ECG: ECG 1")

# Full time domain
plt.subplot(3, 1, 1)
plt.title("Time Domain: FIR vs Adaptive LMS Filtered ECG")
plt.plot(t, ecg_fir, alpha=0.7, label="FIR Filtered ECG")
plt.plot(t, ecg_lms, alpha=0.8, label="Adaptive LMS Filtered ECG")
plt.xlim(0, n / sample_rate)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="upper right")
plt.grid()

# Zoomed single heartbeat (PQRST check)
plt.subplot(3, 1, 2)
plt.title("Zoomed Heartbeat (PQRST Complex): FIR vs Adaptive LMS Filtered ECG")
plt.plot(t - group_delay, ecg_fir, alpha=0.8, label=f"FIR Filtered ECG (delay-aligned by {
    group_delay:.1f} s)")
plt.plot(t, ecg_lms, alpha=0.8, label="Adaptive LMS Filtered ECG")
plt.xlim(beat_start, beat_end)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="lower right")
plt.grid()

# Linear frequency spectrum
plt.subplot(3, 1, 3)
plt.title("Frequency Spectrum: FIR vs Adaptive LMS Filtered ECG")
plt.plot(freqs, fir_fft[:n // 2], alpha=0.7, label="FIR Spectrum")
plt.plot(freqs, lms_fft[:n // 2], alpha=0.8, label="Adaptive LMS Spectrum")
plt.xlim(0, sample_rate / 2)
```

```
plt.ylim(0, 3e-5)
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude (V)")
plt.legend(loc="upper right")
plt.grid()

plt.tight_layout()
plt.show()
```

C.7 heartbeat.py

```
#5 - simulating real time heartbeat detection using fir_filter

import numpy as np
import matplotlib.pyplot as plt

from fir_design import fir_design
from fir_filter import FIRfilter
from fir_realtime_sim import BASELINE_WANDER, MAINS_AND_HF_HUM
from support_funcs import import_ecg_data


# Load and format ECG data
t, ecg, sample_rate = import_ecg_data("ecg1.tsv")  # ECG to analyse
_, template_ecg, _ = import_ecg_data("ecg2.tsv")  # other lab partner's ECG


#Apply filter from Task 3
h = fir_design(sample_rate, [BASELINE_WANDER, MAINS_AND_HF_HUM])

#Extract QRS from TEMPLATE
template_fir = FIRfilter(h)
template_clean = np.zeros(len(template_ecg))
for i in range(len(template_ecg)):
    template_clean[i] = template_fir.dofilter(template_ecg[i])


#Cut around the R peak (include QRS). R is chosen after the cleaning filter's buffer has filled
R = 2576 #sample number of R peak in template ECG (ecg2.tsv)
template = template_clean[R-12:R+13] #25 samples
matched_fir = FIRfilter(np.flip(template)) #reverse coefficients in time
ecg_fir = FIRfilter(h) #fresh cleaning filter for ECG

#matched filter output, stored for plotting
matched_fir_out = np.zeros(len(ecg))

# --- Realtime detection ---
THRESHOLD = 1.5e-11 #from matched filter plot
TRANSIENTS = len(template) + len(h) # ignore output until both filter buffers are full (cleaning
     + matched filter)
MIN_BPM = 30
MAX_BPM = 200
last_beat = None #index of previous beat
heart_rate = []
beat_idx = [] #sample index of each beat with an accepted rate

for i in range(len(ecg)):
    matched_fir_out[i] = matched_fir.dofilter(ecg_fir.dofilter(ecg[i])) ** 2 #square to make
    positive and emphasise peaks
    if i <= TRANSIENTS:
        continue
    if matched_fir_out[i] > THRESHOLD and matched_fir_out[i-1] <= THRESHOLD:
        peak = i
```

```
        if last_beat != None:
            #Ensure heartrate is within reasonable range (30-200 bpm)
            momentary_rate = 60* sample_rate / (peak - last_beat) #bpm
            in_range =  MIN_BPM < momentary_rate < MAX_BPM #Heuristic 1

            prev_rate = heart_rate[-1] if heart_rate else momentary_rate #prevent IndexError
            consistency = 0.75*prev_rate < momentary_rate < 1.25*prev_rate #Heuristic 2
            if in_range and consistency:
                heart_rate.append(momentary_rate)
                beat_idx.append(peak)
        last_beat = peak

print(f"Heart rates calculated: {len(heart_rate)}, mean heart rate: {np.mean(heart_rate):.1f} bpm")

# --- PLOTS ---
delay = (len(h) // 2 + len(template) // 2) / sample_rate # approx. delay added by cleaning + matched filter
template_ms = (np.arange(len(template)) - len(template) // 2) / sample_rate * 1000 # time relative to R peak

plt.figure(figsize=(12, 8))
plt.suptitle("Matched Filter R-Peak Detection")

# Template (matched filter coefficients before time reversal)
plt.subplot(2, 1, 1)
plt.title(f"QRS Template from ECG 2 ({len(template)} samples)")
plt.plot(template_ms, template, marker=".")
plt.xlabel("Time relative to R peak (ms)")
plt.ylabel("Amplitude (V)")
plt.grid(True)

# Squared matched filter output with threshold and detected beats
plt.subplot(2, 1, 2)
plt.title("Squared Matched Filter Output: ECG 1")
plt.plot(t, matched_fir_out, label="Squared matched filter output")
plt.axhline(THRESHOLD, color="red", linestyle="--", label=f"Threshold ({THRESHOLD:.1e})")
plt.plot(t[beat_idx], matched_fir_out[beat_idx], "x", color="black", label=f"Detected beats ({len(beat_idx)})")
plt.axvspan(0, TRANSIENTS / sample_rate, color="grey", alpha=0.2, label="Buffer fill (ignored)")
plt.xlim(0, len(ecg) / sample_rate)
plt.ylim(0, 0.5e-10)
plt.xlabel(f"Time (s), output delayed by {delay:.2f} s")
plt.ylabel("Amplitude (V$^4$)")
plt.legend(loc="upper right")
plt.grid(True)

plt.tight_layout()

# Momentary heart rate, shifted back by the filter delay so times match the ECG
plt.figure(figsize=(12, 4))
plt.title(f"Momentary Heart Rate: ECG 1 (mean {np.mean(heart_rate):.1f} bpm)")
plt.plot(t[beat_idx] - delay, heart_rate, marker=".")
plt.xlim(0, len(ecg) / sample_rate)
plt.xlabel("Time (s)")
plt.ylabel("Heart rate (bpm)")
plt.grid(True)

plt.tight_layout()

plt.show()
```